

# Implementace systémové telemetrie

Lukáš Plachý

28. listopadu 2025

## 1. Cíl práce

Cílem je rozšířit minimalistické jádro systému OS pro RPI0 o modul telemetrie, který umožňuje sledovat běhové statistiky systému:

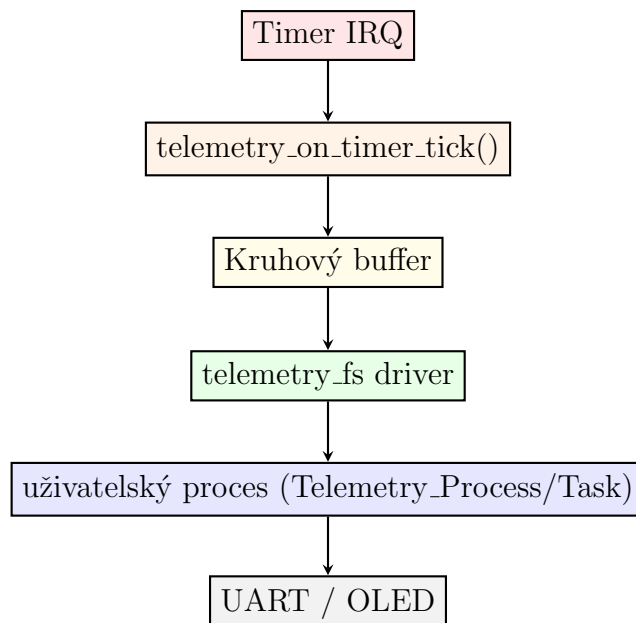
- počet systémových volání za jednotku času,
- počet přerušení,
- počet uzamčení mutexů,
- relativní vytížení plánovače.

Telemetrie musí být implementována přímo v jádře, ale zároveň volitelná pomocí kompilačního přepínače `ENABLE_TELEMETRY`. Data musí být přístupná uživatelskému procesu pomocí běžného souborového rozhraní (`open`, `read`, `write`).

## 2. Architektura řešení

Celý systém telemetrie je rozdělen do tří částí:

1. **Sběr dat** – probíhá na úrovni jádra (timer, syscall, přerušení, mutex).
2. **Uložení do kruhového bufferu** – každou sekundu se zaznamenávají součty.
3. **Zpřístupnění přes souborový systém** – speciální driver `telemetry_fs`.



## 2.1. Telemetrická data

Hodnoty se ukládají do kruhových polí o délce 60, reprezentujících poslední minutu. Každou sekundu se inkrementy přičtou do příslušného pole a po zaplnění se index vrací na nulu a přepisuje nejstarší data.

## 3. Kalibrace časovače

Pro správné fungování výpočtů za sekundu bylo nutné určit skutečný počet timer ticků za sekundu generovaných hardwarem. Za tímto účelem byla dočasně zavedena proměnná:

```
static volatile uint32_t g_last_sec_ticks = 0;
```

Načítala se prostřednictvím rozšířeného výpisu v `telemetry_fs`:

```
write_str("Ticks/sec: ");
write_num(snapshot.tick_accumulator);
write_str("\r\n");
```

Po otestování bylo ověřeno, že hodnota je **1000 ticků za sekundu**, a tím bylo možné správně nastavit:

```
set(TELEMETRY_TICKS_PER_SEC 1000 ...)
```

Dočasné ladicí výpisy byly následně odstraněny.

## 4. Změny v systému

### 4.1. CMakeLists.txt

Byla přidána volba pro podmíněné zapnutí telemetrie:

```
option(ENABLE_TELEMETRY "Enable system telemetry" ON)
if(ENABLE_TELEMETRY)
    add_definitions(-DENABLE_TELEMETRY)
    set(TELEMETRY_TICKS_PER_SEC 1000 CACHE STRING
        "Number of timer ticks per second used by telemetry")
    add_definitions(-DTELEMETRY_TICKS_PER_SEC=${
        TELEMETRY_TICKS_PER_SEC})
endif()
```

## 5. Implementace modulu telemetrie

### 5.1. Hlavička telemetry.h

```
#pragma once
#include <hal/intdef.h>

struct telemetry_snapshot {
    uint32_t syscalls_per_min;
    uint32_t interrupts_per_min;
    uint32_t mutex_locks_per_min;
    uint32_t scheduler_load_percent;
};

void telemetry_init();
void telemetry_reset();
void telemetry_on_timer_tick();
void telemetry_increment_syscall();
void telemetry_increment_interrupt();
void telemetry_increment_mutex();
void telemetry_get_snapshot(telemetry_snapshot* snap);
```

## 6. Integrace do jádra

Do jednotlivých částí systému byla přidána volání telemetrie:

- timer.cpp: při každém IRQ volá `telemetry_on_timer_tick()`.
- interrupt\_controller.cpp: `telemetry_increment_interrupt()`.
- process\_manager.cpp: `telemetry_increment_syscall()` při SWI.
- mutex.cpp: `telemetry_increment_mutex()` při zamknutí mutexu.

## 7. Testovací úloha mutex\_task

Pro ověření správného zvyšování počítadla byla vytvořena samostatná uživatelská úloha:

```
int main(int argc, char** argv)
{
    uint32_t m = mutex_create("test");

    while (true)
    {
        mutex_lock(m);        // zde prob hne
        telemetry_increment_mutex()
        mutex_unlock(m);
    }

    close(m);
    return 0;
}
```

Úloha opakovaně zamyká a odemyká mutex a generuje tak vysoký počet událostí, které musí být v telemetrii viditelné.

## 8. Filesystem driver telemetry\_fs

### 8.1. Hlavička telemetry\_fs.h

```
#pragma once
#include <fs/filesystem.h>
#include "telemetry.h"

class CTelemetry_File : public IFile {
public:
    uint32_t Read(char* buffer, uint32_t num) override;
    bool Close() override { return true; }
};

class CTelemetry_FS_Driver : public IFilesystem_Driver {
public:
    void On_Register() override {}
    IFile* Open_File(const char* path, NFile_Open_Mode mode)
        override;
};

extern CTelemetry_FS_Driver fsTelemetry_FS_Driver;
```

### 8.2. Chování driveru

- FS driver je zaregistrován do pole ovladačů:

```
{ "TELEM_FS", "DEV:telemetry", &fsTelemetry_FS_Driver },
```

- Každé volání Read() načte aktuální snapshot telemetrie přes telemetry\_get\_snapshot().

## 9. Uživatelský proces telemetry\_task

Součástí userspace je také samostatný task pro kontinuální výpis telemetrických dat buď na UART, nebo na OLED displej:

```
uint32_t telem = open("DEV:telemetry", Read_Only);

#if USE_UART_OUTPUT
    // vystup na UART
#else
    // vystup na OLED
#endif

while (true)
{
    uint32_t n = read(telem, buf, sizeof(buf) - 1);
    buf[n] = '\0';

    write_output(buf);

    // kratka pauza
    for (volatile int i = 0; i < 0x100000; i++);
}
```

### 9.1. OLED vs. UART výstup

OLED displej (128×32) je poměrně malý, takže plný výpis telemetrie se na něj nevejde. Proto sloužil zejména jako **rychlý kontrolní výstup** v situacích, kdy se uživatel nechtěl přepojovat na UART.

UART zůstává primárním kanálem pro standardní lidsky čitelné reporty.

## 10. Ukázka výsledného výpisu

Výpis ze souborového systému může vypadat takto:

```
Syscls/min: 160
Int/min: 5390
MtxLck/min: 1200
Scheduler: 99 %
```